

Learning Agile Locomotion on Risky Terrains

Chong Zhang¹, Nikita Rudin¹, David Hoeller¹, Marco Hutter¹

Abstract—Quadruped robots have shown remarkable mobility on various terrains through reinforcement learning. Yet, in the presence of sparse footholds and risky terrains such as stepping stones and balance beams, which require precise foot placement to avoid falls, model-based approaches are often used. In this paper, we show that end-to-end reinforcement learning can also enable the robot to traverse risky terrains with dynamic motions. To this end, our approach involves training a generalist policy for agile locomotion on disorderly and sparse stepping stones before transferring its reusable knowledge to various more challenging terrains by finetuning specialist policies from it. Given that the robot needs to rapidly adapt its velocity on these terrains, we formulate the task as a navigation task instead of the commonly used velocity tracking which constrains the robot’s behavior and propose an exploration strategy to overcome sparse rewards and achieve high robustness. We validate our proposed method through simulation and real-world experiments on an ANYmal-D robot achieving peak forward velocity of ≥ 2.5 m/s on sparse stepping stones and narrow balance beams. Video: youtu.be/Z5X0J8OH6z4

I. INTRODUCTION

Quadrupedal robots require perceptive locomotion abilities on rough and complex terrains, including risky terrains with sparse footholds such as stepping stones and balance beams. Existing works typically employ model-based controllers [1]–[6] which perform well in simulation or under laboratory conditions. However, they are empirically difficult to deploy in the wild due to model mismatch and unexpected slippage [7], [8], and typically require dedicated design (e.g. motion references [9] and handcrafted heuristics such as terrain roughness and pose safety [3]) to achieve specified motions. Furthermore, the dynamics model needs to be simplified especially for the limbs [2], [4], and replanning is often slow due to the heavy computational burden [3], [5].

On the other hand, model-free reinforcement learning (RL)-based methods demonstrate excellent robustness in the wild via domain randomization and massive training data [10], [11]. However, no existing works have used end-to-end RL to handle the sparse footholds on highly risky terrains, where precise motions are necessary to avoid failures, and exploration is difficult due to the numerous ways to fail. First attempts to solve such locomotion challenges using end-to-end learning show promising results, but still have limited terrain complexity [12] or need to exactly know which stones to step on [13]. Another work trains decoupled policies for foothold planning and execution [14], but it cannot achieve dynamic locomotion on risky terrains, and

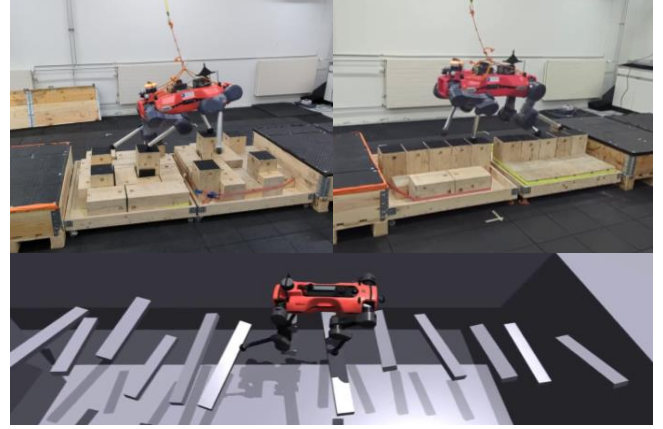


Fig. 1. The ANYmal-D robot executing the learned RL policies on stepping stones, balance beams with gaps, and stepping beams. **Left top:** the stepping stones were 20 cm wide taking up only $\sim 10\%$ of the whole area, and the height difference between high and low stones was 12 cm. The robot reached the peak forward velocity of 2.7 m/s when traversing the stones from left to right. **Right top:** the terrain consisted of two beams of $100\text{ cm} \times 20\text{ cm}$ and three gaps. The peak forward velocity of the robot was 2.5 m/s from right to left. **Bottom:** The widths of stepping beams varied in [12 cm, 17 cm], the horizontal distances between two beams varied in [30 cm, 60 cm], and the vertical distances varied in [0 cm, 20 cm]. For practical reasons, we only tested the policy in simulation, and the peak forward velocity of the robot reached 1.5 m/s from left to right.

decoupled planning and control can inherently limit the range of behaviors that can be expressed [15].

Some existing works combine RL and model-based controllers. For example, [16] uses RL to generate desired foot positions which are then tracked by a whole-body motion controller, as well as corrective joint torques to achieve domain adaptive tracking. Another work uses RL to generate the desired acceleration of the base and tracks it with model-based controllers [17]. All of these approaches, however, demonstrate limited agility with velocity $\leq 0.5\text{ m/s}$. Another recent work [18] opens a new venue by tracking trajectories from a model-based planner with an RL policy and demonstrates remarkable performance, but the motions are constrained by the planner and cannot recover from failures.

In this paper, we explore how dynamic locomotion of a quadrupedal robot can be achieved on highly risky terrains through end-to-end RL. We first train a generalist policy to learn reusable sensorimotor skills on sparse and disorderly stepping stones. Then, we finetune multiple specialist policies from the generalist policy on various more challenging terrains, and successfully transfer two of the specialist policies to real-world stepping stones and narrow beams. We achieve highly agile motions with peak forward velocity ≥ 2.5 m/s, as shown in Fig. 1.

¹ Authors are with the Robotic Systems Lab, ETH Zurich, 8092 Zurich, Switzerland. Emails: {chozhang, rudinn, dhoeller, mahutter}@ethz.ch

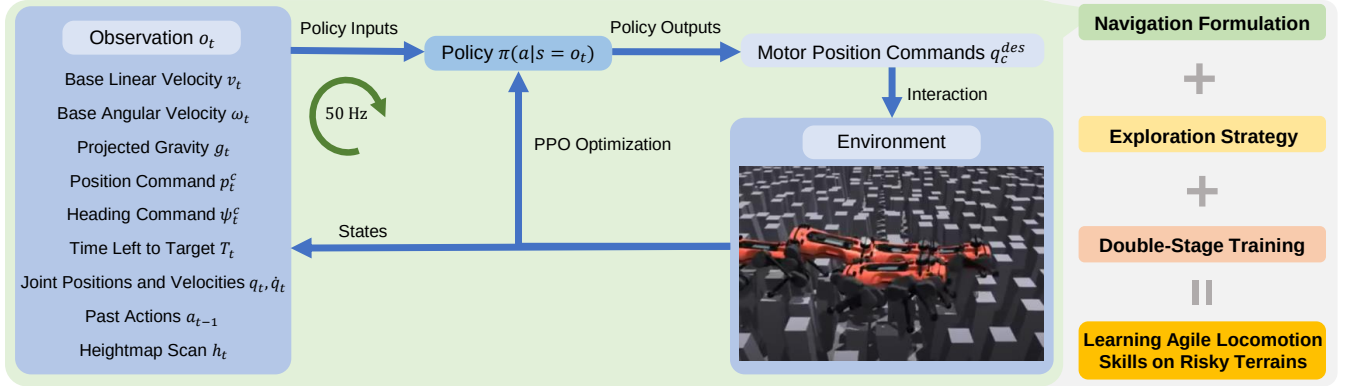


Fig. 2. An overview of our methodology. The learning system is illustrated on the left. Different components are listed on the right.

Traditional learning methods based on the commonly used velocity tracking formulation (e.g., the one used in [19] and [12]) can not achieve these challenging tasks. It is intractable to assign appropriate velocity commands to the robots on risky terrains especially when they need to rapidly adapt their velocities, and the feasible solutions are quite rare and hard to explore compared to nonrisky terrains. Hence, the key idea in this paper is to formulate the task as a navigation problem as in [20], though the risky environments and the sparse rewards make the exploration even harder.

To overcome the hard exploration in the navigation formulation, we improve the curriculum learning design in [20], and incorporate curiosity-driven intrinsic rewards [21]. We also employ a symmetry-based data augmentation from [22], utilizing the left-right and front-back symmetry of our robot to improve the data efficiency. These improvements result in an exploration strategy that can learn a robust generalist policy.

Still, we find such an exploration strategy above is not enough to learn specialist policies from scratch on the terrains in Fig. 1. Besides, we want the learned sensorimotor skills to be reusable for new terrains. To this end, we divide the training procedure into two stages, where the first stage is to learn a common generalist policy on sparse and disorderly stepping stones, and the second stage is to finetune different specialist policies on different terrains from this generalist policy. By doing so, the generalist policy can learn some basic motion skills that are reusable on different terrains, and the specialist policies can benefit from the pre-training of the first stage. This generalist-specialist training procedure enables learning of agile locomotion skills which are then transferred to a real robot on risky terrains.

It is also worth mentioning that we do not focus on perception in this paper and therefore use a ground truth map and a motion capture system for state estimation during real-world deployment. That said, it is possible to use onboard sensors with the techniques proposed in [23].

In summary, our contributions include:

- 1) The extension of the navigation formulation in [20] to challenging risky terrains;
- 2) An exploration strategy consisting of a curriculum, intrinsic rewards, and a symmetry-based data augmen-

tation;

- 3) The methodology of training a generalist policy and finetuning specialist policies under hard exploration;
- 4) Successful sim-to-real transfer of the learned agile motions on challenging stepping stones and balance beams.

II. METHODOLOGY

All of the implementations in this paper are based on the PPO algorithm [24] with massively parallel simulation [19] enabled by the GPU-based simulator Isaac Gym [25]. An overview is presented in Fig. 2.

A. Formulating Locomotion as Navigation

In this paper, we formulate the problem as a navigation task: the robot should get close to an assigned target on the terrain to get high task rewards. The navigation formulation has shown great potential when velocity commands cannot be easily assigned, e.g., when the robot walks in a maze [26] or needs to first recover from a fall and then walk [27]. In [20], advanced locomotion skills like jumping over large gaps and climbing high boxes are learned with the navigation formulation, where velocity-based tracking fails to achieve the desired behaviors.

In this work, considering that the robot should adapt its velocity to the terrain and inappropriate velocity commands can be misleading, we also use the navigation formulation. We deploy the system illustrated in Fig. 2 based on [20], and assign position and heading commands to the robots. Tracking these commands dominates the rewards as follows:

$$r_{\text{task}} = \begin{cases} \frac{c_{\text{task}}}{T_r} \cdot \frac{1}{1 + \|\chi_t - \chi_t^*\|^2}, & \text{if } t > T - T_r \\ 0, & \text{otherwise,} \end{cases} \quad (1)$$

where χ and χ^* are the current and target positions (or headings) of the base, t is the current time, T is the maximum episode length, and T_r defines the duration of the reward. By doing so, the policy is free to modulate the movement of the robot as long as it reaches the target towards the end of the episode.

B. Exploration Strategy

Our exploration strategy improves the prior work [20] in three aspects: the curriculum learning design, the intrinsic

rewards, and the data augmentation. The proposed techniques can empirically train a more robust policy under sparse rewards, and are experimentally analyzed in the ablation studies in Sec. IV-B.

1) *Curriculum Design*: Curriculum learning is a common technique for learning locomotion skills on challenging terrains [7], [10], [19], [20]. In Isaac Gym, a typical implementation of curriculum learning is to generate all terrains of different difficulty levels before training, and distribute the robots on different terrains [19], [20]. Based on this, the curriculum design in [20] is formulated as: 1) go to a higher difficulty level if the policy performs well, 2) go to a lower difficulty level if the policy performs poorly, and 3) go to a random difficulty level if the policy performs well on the highest difficulty level.

However, we find that such a curriculum suffers from sparse rewards and difficulty gaps between levels due to the highly challenging and risky terrains. A policy performing well on a low-difficulty terrain can hardly work on a high-difficulty terrain, leading to a large reward difference that can be incorrectly attributed by the difficulty-agnostic optimizer to different actions, thereby misguiding the optimization. To alleviate this problem, we propose to relax the curriculum update mechanism: the robot only gets demoted when the policy cannot outperform random actions in the remaining distance to the target. By doing so, the robot can keep interacting with the same level until it overcomes it.

2) *Intrinsic Curiosity Rewards*: On top of the task rewards which incentivize the robot to reach the target, we also use curiosity-based intrinsic rewards that encourage the robot to explore novel state-action pairs during training. Among many variants [21], [28], [29], we use the random network distillation (RND) [21] with the version in [30] due to its simple implementation and competitive performance against other variants. Specifically, the curiosity reward is defined as

$$r_{\text{curio}}(s, a) = c_{\text{curio}} \cdot |M_1(s, a) - M_2(s, a)|, \quad (2)$$

where M_1 and M_2 are two randomly initialized neural networks taking state-action (s, a) pairs as inputs, and M_1 can be trained while M_2 is frozen. By fitting M_1 towards the outputs of M_2 with visited state-action pairs, the curiosity rewards will be high for unfamiliar pairs but decay to 0 for familiar ones.

It is worth mentioning that while curiosity rewards are commonly leveraged to improve data efficiency, we do not witness a significant change on the learning curves. Instead, we empirically find that it can improve the robustness of the learned policy, as is discussed in Sec. IV-B. This can potentially be due to an increased coverage of the state space, as suggested by [31] where a larger state-space coverage can improve the robustness of some learned locomotion skills.

3) *Symmetry-Based Data Augmentation*: To improve the data efficiency and reduce base inclination during locomotion, we utilize the symmetric design of our robot to augment sampled data as proposed in [22], which is an adaptation of the duplication method in [32]. To be specific, one state-action pair can be augmented to four pairs with left-right and

front-back symmetry. The augmented state-action pairs are assigned the same target values, advantages, and discounted returns as the original pairs. In this way, the actor and critic networks are enforced to learn symmetric actions and values.

C. Double-Stage Generalist-Specialist Training

In this paper, we achieve agile locomotion on diverse challenging terrains with sparse footholds as shown in Fig. 1. However, even with all the techniques presented above, these motions cannot be learned from scratch due to the extremely sparse rewards. To address this, we first train a generalist policy on relatively simple stepping stones (see Sec. III-A). Then, the learned policy and its critic network are finetuned on new terrains, enabling data-efficient learning of agile motions, where the generalist policy can serve as a pre-trained model.

Besides the data efficiency, the generalist policy is shared for different specialist policies, which means that the learned sensorimotor skills are reusable. Therefore, the costly generalist training stage only happens once, and only data-efficient finetuning is needed for new terrains.

III. EXPERIMENTAL SETUP

We create 4 different types of terrains in Isaac Gym, each with 10 difficulty levels. Among them, "Stones-Everywhere" is used for generalist policy training and benchmarking, while the other three ("Stones-2Rows", "Balance-Beams", and "Stepping-Beams") are used for finetuning. The terrains are described in Sec. III-A. Sec. III-B and Sec. III-C present the specific rewards and task-related randomizations of each terrain type respectively.

A. Terrains

The four terrain types are displayed in Fig. 3 and described in the following paragraphs. Since the "Stones-Everywhere" type serves to benchmark the performance of different learning methods, we present it in detail, while other terrain types are introduced briefly.

1) *Stones-Everywhere*: This is the terrain type used to train the generalist policy and benchmark the ablation studies. Each stone is assigned a grid square and has a random height with a random X-Y plane shift from the square center. Highly disorderly and sparse stepping stones can be generated in this way. The parameters for different levels include the stone size w_{stone} , the stone gap w_{gap} , the maximum shift s_{max} , and the maximum height h_{max} . The stone size is the width of the square stones, and the stone gap is the grid size minus the stone size, i.e., the size of gap between two stones with zero shift. Both the X shift and the Y shift are uniformly sampled from $[-s_{\text{max}}, s_{\text{max}}]$, and the stone height is uniformly sampled from $[-h_{\text{max}}, h_{\text{max}}]$.

The parameters are presented in Table I, together with the sparsity representing the ratio of the non-steppable area over the whole terrain.

2) *Stones-2Rows*: Terrains of this type have 1, 2 or 3 rows of stepping stones which match what we can build in the lab. By doing so we can achieve a policy that does not overfit to a specific gait pattern.

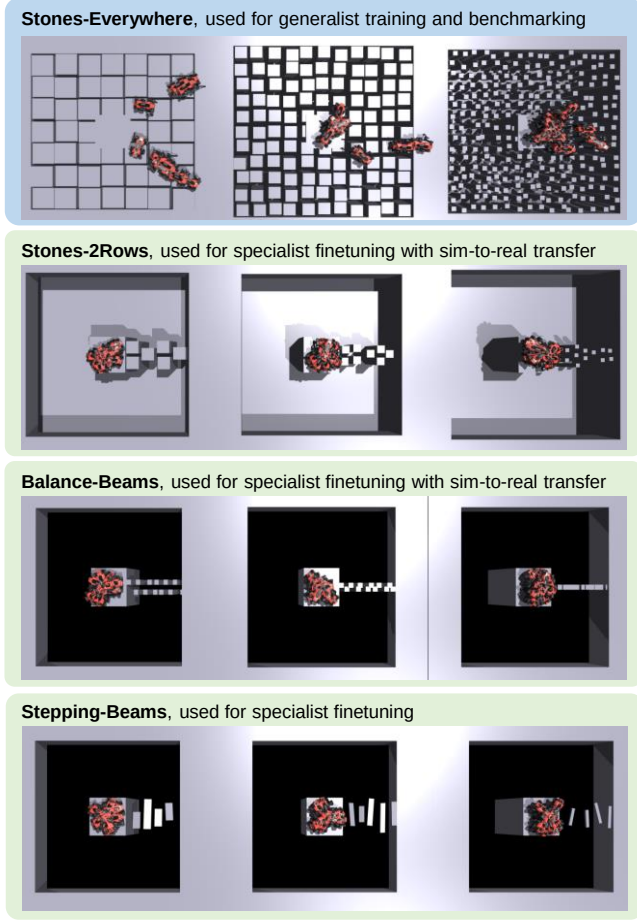


Fig. 3. Different types of terrains with difficulty levels 0, 5, and 9 from left to right. One terrain type can have different subtypes that stress different features, as is explained in Sec. III-A.

3) *Balance-Beams*: Terrains of this type have stepping stones evolving from two rows merging into one as the level goes up. Random gaps are added to the terrains as well. Such evolving stones can guide the policy to learn catwalk-like motions. Beams with gradually smaller widths can be a more intuitive choice, but this can lead to infeasible crawling motions in Fig. 4, a local minimum that works only on wide beams.

4) *Stepping-Beams*: Terrains of this type have a sequence of beams to step on, and the beams are more narrow and randomized on higher levels. This terrain type is hard to build in the lab so we only tried it in simulation to show that finetuning can work on a wide range of different terrains.

B. Rewards

The same reward function can work on all terrains in simulation, yet slight terrain-dependant modifications of are necessary for elegant motions in the real world. We present the rewards in Table IV in the Appendix, and the changes in weights for special tasks are annotated. For each task on its terrain, the reward function is the sum of all terms.

Among these rewards, we have position tracking and heading tracking for task performance. We use penalties for early termination and collision. We also regularize the joint velocities, the acceleration of the base and feet, the action

TABLE I
PARAMETERS OF "STONES-EVERYWHERE" TERRAINS

Difficulty Level	Stone Size w_{stone} (cm)	Stone Gap w_{gap} (cm)	Max. Shift s_{max} (cm)	Max. Height h_{max} (cm)	Sparsity (%)
0	92	8	3.6	1	15.4
...	...	...	...	...	...
5	52	18	8.2	6	44.8
...	...	...	...	...	...
9	20	26	11.8	10	81.1

* All parameters except the calculated sparsity are linear to the levels so we only list the levels 0, 5, and 9.

** For reference, [12] built the stepping stones of $\sim 65\%$ sparsity without random heights and significant shifts.



Fig. 4. The learned infeasible crawling motions on balance beams due to inappropriate terrain curriculum design. The terrain here for display is far less challenging than the terrains in the real world, but the robot cannot traverse it. When the beam is narrow, the robot cannot find its next step on the beam.

rate, the torques, and the contact forces. We guide exploration with the "Don't wait" and "Move in Direction" rewards as in [20], and provide a "Stand Still" rewards to make the robot stand still after reaching the target.

We regularize the aggressive motions on "Stones-2Rows" and "Balance-Beams", and further regularize the torques on "Balance-Beams". We also add a "Stand Pose" reward to "Stones-2Rows" and "Balance-Beams" to prepare the robot for further repetitive tests.

C. Domain Randomization

We use domain randomization to improve robustness and facilitate sim-to-real transfer [33], [34]. Task-related randomization settings are presented in Table III in the Appendix. We slightly extend the episode length to slow down the aggressive motions on "Stones-2Rows" and "Balance-Beams". The center of mass (CoM) is heavily randomized for "Balance-Beams" to reduce the dangerous shaking of the base on the beams.

IV. RESULTS IN SIMULATION

A. Learned Motions in Simulation

Our learned policy demonstrates many interesting behaviors on "Stones-Everywhere" in simulation. In Fig. 5, we made the robot walk around on stepping stones by providing a series of waypoints as target positions and headings, showcasing the ability of omnidirectional locomotion learned by our method. During locomotion, the robot also had different gait patterns and contact modes under different situations.

We also find the policy can recover from missed steps in simulation, as is displayed in Fig. 6. The ~ 50 kg robot is able to quickly react to the external force disturbances and continues to walk even when the forces on all feet reaches ~ 100 N.

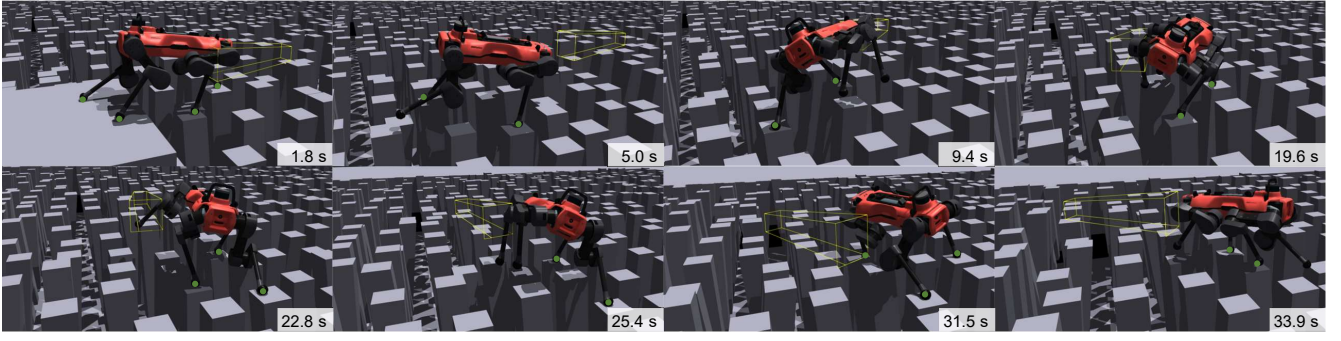


Fig. 5. The robot walking around on the hardest “Stones-Everywhere” terrain in simulation. The yellow boxes indicate a series of target positions and headings we provided to the robot to achieve omnidirectional locomotion. The robot had different gait patterns under different situations, e.g., the 3-contact walk or 2-contact trot gaits when easy to find safe footholds ahead, and the 2-contact pace gaits otherwise. Feet in contact with the terrain are marked.

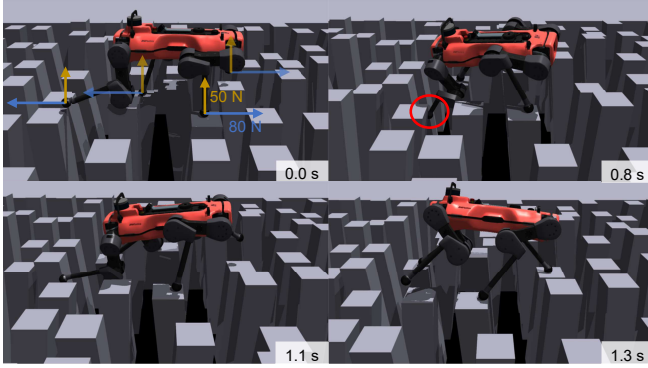


Fig. 6. The robot could dynamically recover from missed steps due to external force disturbances while walking on stepping stones in simulation. **Left top:** the robot received external forces on its four feet lasting 0.4 s when traversing the stones from left to right. **Right top:** the robot missed a step due to the external forces. **Left bottom:** the robot recovered from the missed step by pulling itself upwards and forwards with the legs pressing the stones. **Right bottom:** the robot continued to walk after the missed step.

B. Benchmarked Ablation Studies

We compare different settings and analyze the robustness of the learned policies on “Stones-Everywhere”. The compared settings are:

- 1) w/o navigation: where the velocity tracking formulation is used based on [19] with the corresponding curriculum, while the intrinsic rewards and the symmetry-based data augmentation are also applied;
- 2) w/o curriculum: every technique proposed in this paper except the improved curriculum is used, and the curriculum is the one in [20];
- 3) w/o curiosity: every technique proposed in this paper except the intrinsic curiosity rewards is used;
- 4) w/o augmentation: every technique proposed in this paper except the data augmentation is used;
- 5) proposed: every technique proposed in this paper is used.

Two metrics are used for the comparison of performance and robustness. The first one is the success rate of traversing 6 meters on different difficulty levels, as is shown in Fig. 7. After level 4 with sparsity $\sim 40\%$, the averaged success rate for velocity tracking is lower than 90% and quickly drops down as the difficulty level goes up. In comparison,

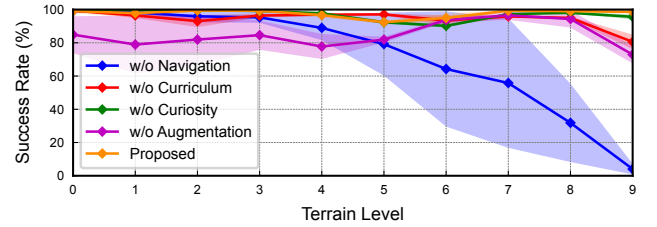


Fig. 7. Success rate of traversing 6 meters on different levels of “Stones-Everywhere”. The settings based on the navigation formulation far outperform the velocity tracking one. Three runs (policies trained with different random seeds) per setting were performed to calculate the mean (line) and std (in shadow) values, and the checkpoints of 8000 iterations (all converged) were used. The velocity command used for testing velocity tracking is 0.8 m/s forward, which demonstrates the highest robustness empirically in simulation. Each policy’s success rate is calculated from 1000 trials on 100 diverse randomized terrains.

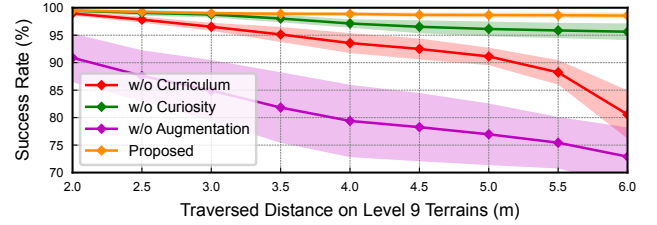


Fig. 8. Success rate of traversing different distances on the hardest level of “Stones-Everywhere”. Our proposed strategy stably learns robust policies over long distances. The shadow indicates the std values.

the navigation formulation can achieve a high success rate, though the “w/o augmentation” setting does not perform well on low-difficulty terrains. This can be explained by the policy’s inability to overcome the hardest terrains without data augmentation, and the neural network policy forgets the samples on low-difficulty terrains when the robots get stuck on high-difficulty ones.

The second metric is the success rate of traversing different distances on the highest difficulty level, which demonstrates a clearer trend between the navigation-formulated ones, as is shown in Fig. 8. All techniques in our strategy significantly contribute to improving robustness, with symmetry-based data augmentation being the most critical one. The improved curriculum design and the curiosity rewards have relatively smaller effects. Besides, the symmetry-based data augmentation also makes the motions more balanced with less base inclination.

		Test Terrains			
		Stones- Everywhere	Stones- 2Rows	Balance- Beams	Stepping- Beams
Training Terrains	Stones- Everywhere	9/9	6/9	<0/9	2/9
	Stones- 2Rows	9/9	9/9	3/9	6/9
	Balance- Beams	3/9	<0/9	9/9	<0/9
	Stepping- Beams	4/9	7/9	<0/9	9/9

Fig. 9. The transferability matrix of our learned policies. Values (before “/9”) in the matrix are the maximum difficulty (0 ~ 9 with 9 being the hardest) of test terrains that can be traversed for 3.5m with an 80% success rate. For the “<0”s, the success rate is within [50%, 80%) on level 0.

C. Finetuning v.s. From Scratch

We train specialist policies on multiple challenging terrains as shown in Fig. 1 and 3 by finetuning the learned generalist policy. The generalist policy took 8000 iterations, the “Stones-2Rows” policy took 2000 iterations, the “Balance-Beams” policy took 3000 iterations, and the “Stepping-Beams” policy took 1000 iterations. In comparison, when the specialist policies are trained from scratch for 10000 iterations per task, the robot can not learn to traverse the terrains, as we design these terrains to be quite challenging.

D. Transferability of Policies

We also test whether policies trained on different terrains are transferable to other terrains. Figure 9 presents the transferability matrix of our learned policies. Due to the similarity of terrains, policies trained on “Stones-Everywhere” and “Stones-2Rows” can be mutually transferred, and the latter is also transferrable to and from “Stepping-Beams”. For other terrain pairs, the policies do not transfer well. Such results indicate that the learned policies highly differentiate due to their training data although they originate from the same generalist policy. Hence, if we want a unified policy that performs well on various terrains, we must randomize all possible terrains the robot may encounter. Otherwise, the learned policy cannot generalize well.

V. REAL-WORLD RESULTS

Due to practical reasons, we only build two terrains in the real world: stepping stones and balance beams, and successfully transfer the learned “Stones-2Rows” and “Balance-Beams” policies. To minimize the sim-to-real gap and focus on the locomotion problem, we use a motion capture system to get the robot’s state and terrain information.

A. Agile Motions

Motions of the learned policies on real terrains are displayed in Fig. 10 and 11. The motions involve multiple leaps and rapidly changing velocities which are difficult to achieve with velocity tracking. To solve the task in the given episode length, the robot takes different strategies on

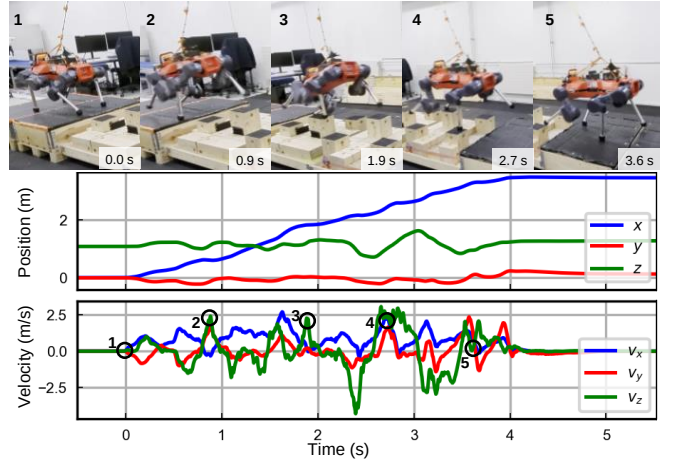


Fig. 10. Learned agile motions on stepping stones in real-world experiments. The robot frequently leaps upwards and forwards to traverse the terrain. The robot positions are sampled from the motion capture system in the world frame, and the velocities are obtained by differentiating the positions.

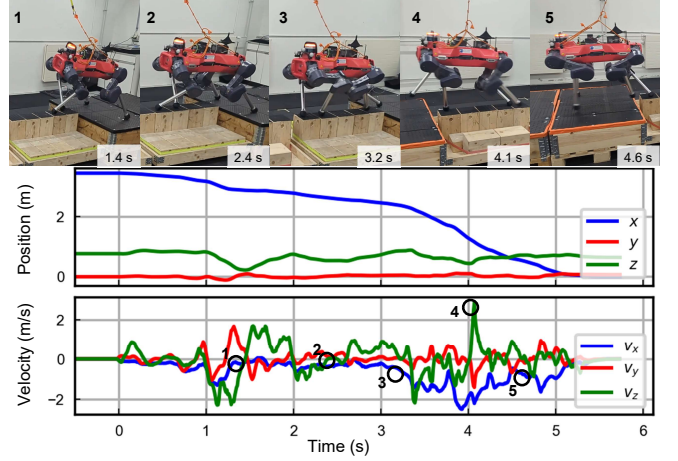


Fig. 11. Learned agile motions on balance beams in real-world experiments. The robot first slowly aligns itself with the beam center before landing on the beam. Afterward, the robot rapidly alternates its left and right diagonals, and makes a speed burst (3 ~ 5 s in the plot) to traverse the beams.

TABLE II

REPETITIVE ROBUSTNESS TEST RESULTS ON REAL TERRAINS

Real Terrain	Stones-2Rows	Balance-Beams
Success Rate	8/10	5/5
Survival Rate	10/10	5/5

different terrains. It keeps leaping on stepping stones, while first slowly aligning itself towards the center and then making a speed burst to quickly traverse the terrain on balance beams.

B. Robustness Tests

We did repetitive robustness tests for the two policies (see also the video attachment), and the results are presented in Table II. The success rate counts the trials that were perfectly finished, while the survival rate also counts the cases where the robot missed a step but recovered.

Our method also differs from the model-based controllers in that the robot can quickly react to unexpected missed steps and survive, as is discussed in Sec. IV-A. We refer readers

to the video attachment to see the emergent behaviors. It is worth mentioning that our controller is vulnerable to mapping drifts, and a 5-cm odometry drift can make the robot fall, though this is partly due to the highly risky terrains.

VI. LIMITATIONS AND DISCUSSION

Though we can achieve agile locomotion on various terrains, we have difficulty training a unified policy for all because the learning progresses on different terrains can vary and interfere with each other. Also, we use slightly different settings for different specialist tasks with human insights to help the sim-to-real transfer. The reward functions can be tedious to tune, and the rare failure cases are hard to interpret.

To be specific, the task reward functions are quite an "equilibrium" of multiple motivations. First, as we want the robot to automatically discover a way to reach the target without human priors, a short reward duration is preferred but can also lead to sparse rewards and hard exploration. Second, a long episode length can make the motions less aggressive for sim-to-real transfer but also makes the task rewards sparse. Hence, tuning the rewards is tedious, and we partly sidestep these trade-offs by pre-training and finetuning in this paper.

Regarding the failure cases, it is always difficult to achieve a 100% success rate, and the rare failure cases do exist. However, contrary to model-based controllers whose failure cases are easy to understand (e.g., missed steps, slippery, or unexpected collisions), our controllers' failure cases are more difficult to interpret. In these rare cases, the robot may misplace its feet in the air when there are feasible footholds, which might be due to the singularity of neural networks [35]. We believe representation learning techniques such as contrastive learning [36] may help solve this.

VII. CONCLUSION AND FUTURE WORKS

In this paper, we achieve end-to-end learning of agile quadrupedal locomotion on challenging risky terrains. To this end, we propose to use the navigation formulation and an exploration strategy with multiple improvements from previous works. We apply two-staged training to overcome exploration on challenging terrains, with specialist policies on different terrains that are finetuned from one generalist policy.

Our future work will include: 1) making the terrain perception onboard based on online representation techniques such as the one in [23]; 2) training a unified policy for locomotion on all kinds of terrains; and 3) making the learned policy more interpretable and robust.

APPENDIX

A. Task Specifications

The task-related domain randomization settings are presented in Table III, and the used symbols and reward terms in Table IV.

TABLE III
DOMAIN RANDOMIZATION SETTINGS IN DIFFERENT TASKS

Parameter	Randomization Range
Stones-Everywhere	
Episode Length (s)	[5, 7]
Target Distance (m)	[1.5, 4.9]
Stones-2Rows	
Episode Length (s)	[6, 8]
Target Distance (m)	[3.5, 4.5]
Heightmap Scan Drift (m)	[-0.05, 0.05]
Balance-Beams	
Episode Length (s)	[8, 10]
Target Distance (m)	[3.5, 4.5]
Heightmap Scan Drift (m)	[-0.025, 0.025]
Center of Mass Bias (m)	x : [-0.15, 0.15] y : [-0.05, 0.05] z : [-0.1, 0.2]
Stepping-Beams	
Episode Length (s)	[5, 7]
Target Distance (m)	[2.5, 4.0]

TABLE IV
USED SYMBOLS AND REWARD FUNCTION TERMS

Symbol	Description
t and T	The current timestep, and the whole length of an episode
p and p^*	The 2-d position and the target position of the robot
ψ and ψ^*	The heading and the target heading of the robot
$q, \dot{q}$, and $\dot{q}_{\text{lim}}$	The joint positions, joint velocities, and the joint velocity limit of the robot
v, v^H , and ω	The linear velocity, horizontal linear velocity, and angular velocity of the robot base
z	The height of the robot base
v_f and F_f	The linear velocity and the contact force of the f -th foot
a	The action, i.e., the target joint positions
τ and τ_{lim}	The torques and the torque limit of the joints
g	The projected gravity on the robot base
$\delta_r(t_0)$	$\frac{1}{t_0} \cdot \mathbb{1}(t > T - t_0)$, the reward duration mask for the last t_0 seconds
$\delta_p(d_0)$	$\mathbb{1}(\ p_t - p_t^*\ < d_0)$, the robot position mask
$\delta_\psi(\theta_0)$	$\mathbb{1}(\ \psi_t - \psi_t^*\ < \theta_0)$, the robot heading mask
"S2"	The abbreviation for "Stones-2Rows"
"BB"	The abbreviation for "Balance-Beams"

Reward Term	Expression	Weight
Position Tracking	$\frac{1}{1 + \ p_t - p_t^*\ ^2}$	$10\delta_r(2)$
	for "S2" and "BB"	$25\delta_r(4)$
Heading Tracking	$\frac{1}{1 + \ \psi_t - \psi_t^*\ ^2}$	$5\delta_p(2)\delta_r(4)$
	for "S2" and "BB"	$12\delta_p(2.5)\delta_r(4)$
Termination Penalty	const for early termination	-200
Collision Penalty	const for thighs and shanks	-1
Joint Velocity Penalty	$\ \dot{q}_t\ ^2$	-0.001
Joint Velocity Limit	$\sum_{i=1}^{12} \max(\dot{q}_{t,i} - 0.9\dot{q}_{\text{lim}}, 0)$	-1
Base Accel. Penalty	$\ \dot{v}_t\ ^2 + 0.02\ \dot{\omega}_t\ ^2$	-0.001
Feet Accel. Penalty	$\sum_{f=1}^4 \ \ddot{v}_{f,t}\ $	-0.0005
Action Rate Penalty	$\ a_t - a_{t-1}\ ^2$	-0.01
Torque Penalty	$\ \tau_t\ ^2$	-0.00001
	for "BB"	-0.00002
Torque Limit	$\sum_{i=1}^{12} \max(\tau_{t,i} - \tau_{\text{lim}}, 0)$	-0.2
	for "BB" w/ $\tau_{\text{lim}} = 0.8\tau_{\text{lim}}$	-0.5
Contact Force Penalty	$\sum_{f=1}^4 \text{clip}(\ F_{f,t}\ - 700, 0, 700)^2$	-0.000025
Don't Wait	$\mathbb{1}(\ v_t\ < 0.2)$	$-(1 - \delta_p(1))$
Move in Direction	$\cos(\angle(v_t, p_t^* - p_t))$	1 for first 150 iter
Stand Still	$2.5\ v_t\ + 1\ \omega_t\ $	$-\delta_p(0.25)\delta_\psi(0.5)\delta_r(1)$
Aggressive Motion	$(\ v_t^H\ - 1)^2 \cdot \mathbb{1}(\ v_t^H\ > 1)$	0
	for "S2" and "BB"	-5
Stand Pose	$ z_t - 0.6 + g_{x,t}^2 + g_{y,t}^2$	0
	for "S2" and "BB"	$-5\delta_p(0.25)\delta_\psi(0.5)\delta_r(1)$

B. Learning Settings

Some learning-related settings are presented in Table V.

TABLE V
LEARNING SETTINGS

Configuration	Values
Actor and critic	MLP, hidden layer size (512, 256, 128) All observations flattened
Exteroception	25 × 16 heightmap, 7cm gridsize
Number of robots	4096
Policy steps per iteration	48
Clip negative rewards to 0?	No
RND M1	MLP, hidden layer size (128, 128)
RND M2	MLP, hidden layer size (256, 256)
RND reward weight	1.0
RND optimization weight	1.0
Loss for RND M1	mean L1-loss

ACKNOWLEDGMENT

This work is supported in part by the EU Horizon 2020 program grant agreement No.852044

REFERENCES

- [1] A. W. Winkler, C. D. Bellicoso, M. Hutter, and J. Buchli, "Gait and trajectory optimization for legged systems through phase-based end-effector parameterization," *IEEE Robotics and Automation Letters*, vol. 3, no. 3, 2018.
- [2] F. Jenelten, R. Grandia, F. Farshidian, and M. Hutter, "Tamols: Terrain-aware motion optimization for legged systems," *IEEE Transactions on Robotics*, vol. 38, no. 6, pp. 3395–3413, 2022.
- [3] S. Fahmi, V. Barasuol, D. Esteban, O. Villarreal, and C. Semini, "Vital: Vision-based terrain-aware locomotion for legged robots," *IEEE Transactions on Robotics*, 2022.
- [4] R. Grandia, F. Jenelten, S. Yang, F. Farshidian, and M. Hutter, "Perceptive locomotion through nonlinear model-predictive control," *IEEE Transactions on Robotics*, 2023.
- [5] O. Melon, R. Orsolino, D. Surovik, M. Geisert, I. Havoutis, and M. Fallon, "Receding-horizon perceptive trajectory optimization for dynamic legged locomotion with learned initialization," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 9805–9811.
- [6] F. Jenelten, T. Miki, A. E. Vijayan, M. Bjelonic, and M. Hutter, "Perceptive locomotion in rough terrain—online foothold optimization," *IEEE Robotics and Automation Letters*, vol. 5, no. 4, 2020.
- [7] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, "Learning quadrupedal locomotion over challenging terrain," *Science robotics*, vol. 5, no. 47, p. eabc5986, 2020.
- [8] F. Jenelten, J. Hwangbo, F. Tresoldi, C. D. Bellicoso, and M. Hutter, "Dynamic locomotion on slippery ground," *IEEE Robotics and Automation Letters*, vol. 4, no. 4, 2019.
- [9] Z. Zhou, B. Wingo, N. Boyd, S. Hutchinson, and Y. Zhao, "Momentum-aware trajectory optimization and control for agile quadrupedal locomotion," *IEEE Robotics and Automation Letters*, vol. 7, no. 3, 2022.
- [10] T. Miki, J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, "Learning robust perceptive locomotion for quadrupedal robots in the wild," *Science Robotics*, vol. 7, no. 62, p. eabk2822, 2022.
- [11] M. Tranzatto, T. Miki, M. Dharmadhikari, L. Bernreiter, M. Kulkarni, F. Mascarich, O. Andersson, S. Khattak, M. Hutter, R. Siegwart, et al., "Cerberus in the darpa subterranean challenge," *Science Robotics*, vol. 7, no. 66, p. eabp9742, 2022.
- [12] A. Agarwal, A. Kumar, J. Malik, and D. Pathak, "Legged locomotion in challenging terrains using egocentric vision," in *6th Annual Conference on Robot Learning*, 2022.
- [13] Z. Xie, H. Y. Ling, N. H. Kim, and M. van de Panne, "Allsteps: curriculum-driven learning of stepping stone skills," in *Computer Graphics Forum*, vol. 39, no. 8. Wiley Online Library, 2020, pp. 213–224.
- [14] V. Tsounis, M. Alge, J. Lee, F. Farshidian, and M. Hutter, "Deepgait: Planning and control of quadrupedal gaits using deep reinforcement learning," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, 2020.
- [15] Y. Song, A. Romero, M. Müller, V. Koltun, and D. Scaramuzza, "Reaching the limit in autonomous racing: Optimal control versus reinforcement learning," *Science Robotics*, vol. 8, no. 82, p. eadg1462, 2023.
- [16] S. Gangapurwala, M. Geisert, R. Orsolino, M. Fallon, and I. Havoutis, "Rloc: Terrain-aware legged locomotion using reinforcement learning and optimal control," *IEEE Transactions on Robotics*, 2022.
- [17] Z. Xie, X. Da, B. Babich, A. Garg, and M. v. de Panne, "Glide: Generalizable quadrupedal locomotion in diverse environments with a centroidal model," in *International Workshop on the Algorithmic Foundations of Robotics*. Springer, 2023, pp. 523–539.
- [18] F. Jenelten, J. He, F. Farshidian, and M. Hutter, "Dtc: Deep tracking control," *Science Robotics*, vol. 9, no. 86, p. eadh5401, 2024.
- [19] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, "Learning to walk in minutes using massively parallel deep reinforcement learning," in *Conference on Robot Learning*. PMLR, 2022, pp. 91–100.
- [20] N. Rudin, D. Hoeller, M. Bjelonic, and M. Hutter, "Advanced skills by learning locomotion and local navigation end-to-end," in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS 2022)*. IEEE, 2022.
- [21] Y. Burda, H. Edwards, A. J. Storkey, and O. Klimov, "Exploration by random network distillation," in *7th International Conference on Learning Representations*, 2019.
- [22] D. Hoeller, N. Rudin, D. Sako, and M. Hutter, "Anymal parkour: Learning agile navigation for quadrupedal robots," *Science Robotics*, vol. 9, no. 88, p. eadi7566, 2024.
- [23] D. Hoeller, N. Rudin, C. Choy, A. Anandkumar, and M. Hutter, "Neural scene representation for locomotion on structured terrain," *IEEE Robotics and Automation Letters*, vol. 7, no. 4, 2022.
- [24] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.
- [25] V. Makoviychuk, L. Wawrzyniak, Y. Guo, M. Lu, K. Storey, M. Macklin, D. Hoeller, N. Rudin, A. Allshire, A. Handa, and G. State, "Isaac gym: High performance GPU based physics simulation for robot learning," in *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*, 2021.
- [26] E. Chane-Sane, C. Schmid, and I. Laptev, "Goal-conditioned reinforcement learning with imagined subgoals," in *International Conference on Machine Learning*. PMLR, 2021, pp. 1430–1440.
- [27] C. Yang, K. Yuan, Q. Zhu, W. Yu, and Z. Li, "Multi-expert learning of adaptive legged locomotion," *Science Robotics*, vol. 5, no. 49, p. eabb2174, 2020.
- [28] D. Pathak, P. Agrawal, A. A. Efros, and T. Darrell, "Curiosity-driven exploration by self-supervised prediction," in *International conference on machine learning*. PMLR, 2017, pp. 2778–2787.
- [29] Y. Burda, H. Edwards, D. Pathak, A. Storkey, T. Darrell, and A. A. Efros, "Large-scale study of curiosity-driven learning," in *International Conference on Learning Representations*, 2019.
- [30] H. Sun, L. Han, R. Yang, X. Ma, J. Guo, and B. Zhou, "Optimistic curiosity exploration and conservative exploitation with linear reward shaping," in *Advances in Neural Information Processing Systems*, 2022.
- [31] C. Zhang, W. Yu, and Z. Li, "Accessibility-based clustering for efficient learning of locomotion skills," in *2022 International Conference on Robotics and Automation (ICRA)*. IEEE, 2022, pp. 1600–1606.
- [32] F. Abdolhosseini, H. Y. Ling, Z. Xie, X. B. Peng, and M. Van de Panne, "On learning symmetric locomotion," in *Proceedings of the 12th ACM SIGGRAPH Conference on Motion, Interaction and Games*, 2019, pp. 1–10.
- [33] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," in *2017 IEEE/RSJ international conference on intelligent robots and systems (IROS)*. IEEE, 2017, pp. 23–30.
- [34] Z. Xie, X. Da, M. Van de Panne, B. Babich, and A. Garg, "Dynamics randomization revisited: A case study for quadrupedal locomotion," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 4955–4961.
- [35] E. Orhan and X. Pitkow, "Skip connections eliminate singularities," in *International Conference on Learning Representations*, 2018.
- [36] M. Laskin, A. Srinivas, and P. Abbeel, "Curl: Contrastive unsupervised representations for reinforcement learning," in *International Conference on Machine Learning*. PMLR, 2020, pp. 5639–5650.